**PEMROGRAMAN BERBASIS FRAMEWORK**
**JOBSHEET 9**

**NAMA : Muhammad Fahreza Rohmansyah**
**KELAS : 3E**
**NO. ABSEN : 15**
**NIM : 2341720151**

**PROGRAM STUDI TEKNIK INFORMATIKA**
**JURUSAN TEKNOLOGI INFORMASI**
**POLITEKNIK NEGERI MALANG**
**2026**

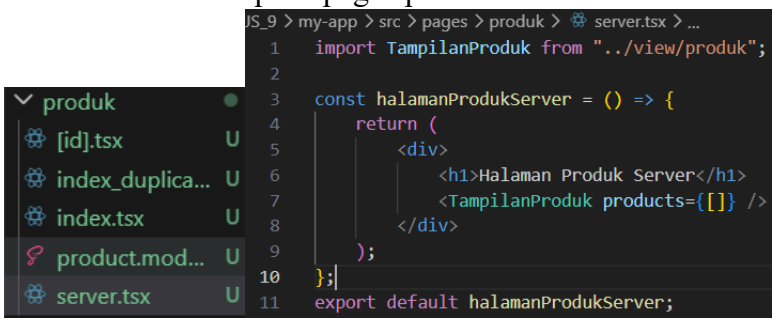
## Link Github:

https://github.com/rezfahreza/PBF_2026.git

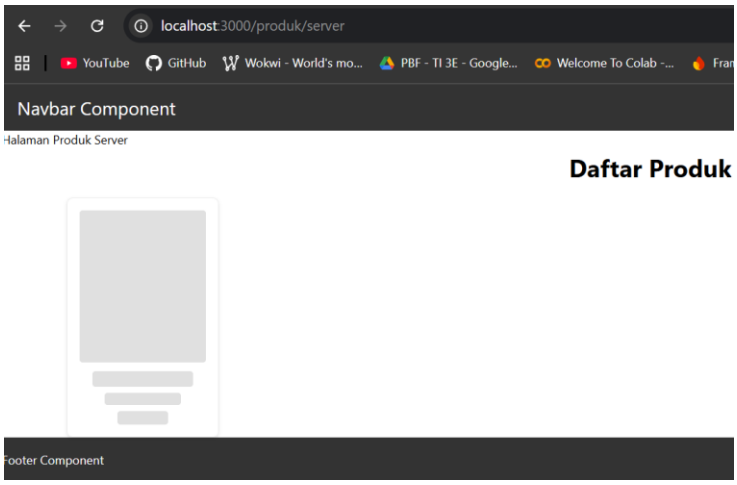
# Praktikum

## 1. Setup Halaman SSR

- Buat file baru pada pages/products/server.tsx lalu modifikasi file server.tsx




- Jalankan browser



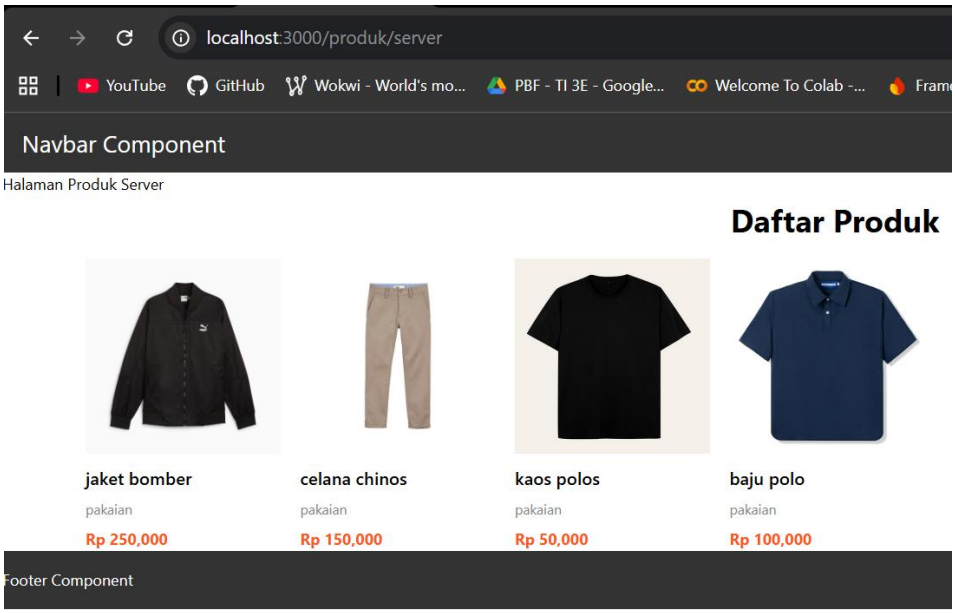

## 2. Implementasi getServerSideProps pada server.tsx

- Menambahkan type produk dan getServerSideProps di server.tsx

JS_9 > my-app > src > pages > produk > server.tsx > getServerSideProps > props > products

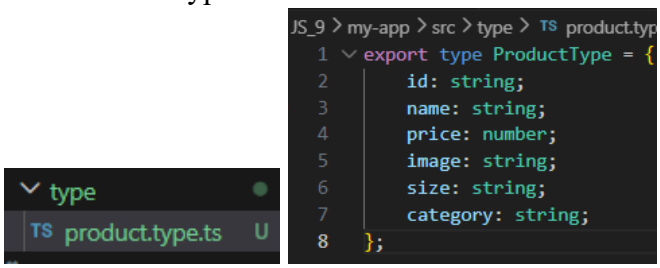
```
import TampilanProduk from "../view/produk";

type ProductType = {
  id: string;
  name: string;
  price: number;
  image: string;
  size: string;
  category: string;
};

const halamanProdukServer = (props: { products: ProductType[] }) => {
    const { products } = props;
    return (
        <div>
            <h1>Halaman Produk Server</h1>
            <TampilanProduk products={products} />
        </div>
    );
};
export default halamanProdukServer;

export async function getServerSideProps() {
    // Simulate an API call to fetch products
    const res = await fetch('http://localhost:3000/api/produk');
    const response = await res.json();

    return {
        props: {
            products: response.data, // Pass the products data to the page component
        }
    };
}
```

- Jalankan browser




## 3. Refactor Type ( produk type )

- Buat folder types pada folder pages dan buat file Product.type.ts lalu modifikasi Product.type.ts




- Modifikasi pada file server.tsx

```
JS_9 > my-app > src > pages > produk > server.tsx > ...
import TampilanProduk from "../view/produk";
import { ProductType } from "../../type/product.type";

const halamanProdukServer = (props: { products: ProductType[] }) => {
    const { products } = props;
    return (
        <div>
            <h1>Halaman Produk Server</h1>
            <TampilanProduk products={products} />
        </div>
    );
};
export default halamanProdukServer;

export async function getServerSideProps() {
    // Simulate an API call to fetch products
    const res = await fetch('http://localhost:3000/api/produk');
    const response = await res.json();

    return {
        props: {
            products: response.data, // Pass the products data to the page component
        }
    };
}
```

# Tugas Praktikum

## 1. Buat 2 halaman:

- /products (CSR)
- /products/server (SSR)

CSR:

```
JS_9 > my-app > src > pages > produk > index.tsx > Kategori
import useSWR from "swr";
import styles from "./product.module.scss";
import fetcher from "../utils/swr/fetcher";
import TampilanProduk from "../view/produk";

type ProductType = {
  id: string;
  name: string;
  price: number;
  image: string;
  size: string;
  category: string;
};

const Kategori = () => {
  const { data, error, isLoading } = useSWR("/api/produk", fetcher);

  const products: ProductType[] = data?.data || [];

  if (error) {
    return <p>Gagal mengambil data</p>;
  }

  return (
    <TampilanProduk products={products} />
  );
};

export default Kategori;
```

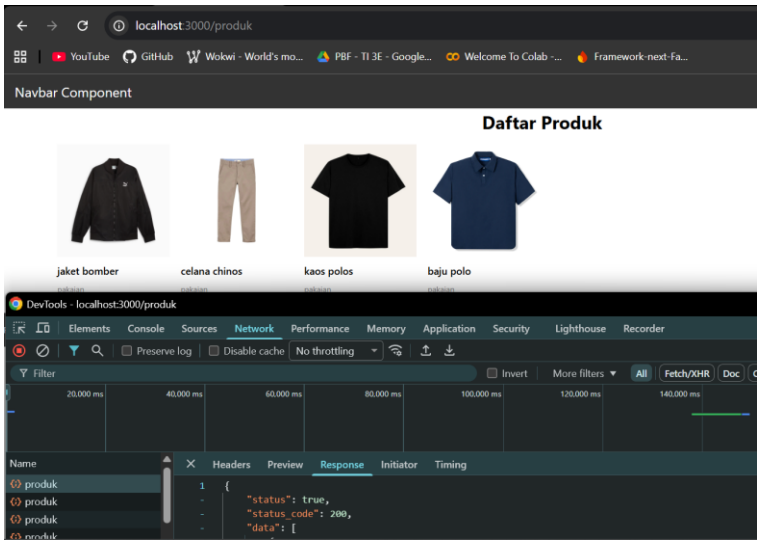
SSR:

```
import TampilanProduk from "../view/produk";
import { ProductType } from "../../type/product.type";

const halamanProdukServer = (props: { products: ProductType[] }) => {
    const { products } = props;
    return (
        <div>
            <h1>Halaman Produk Server</h1>
            <TampilanProduk products={products} />
        </div>
    );
};
export default halamanProdukServer;

export async function getServerSideProps() {
    // Simulate an API call to fetch products
    const res = await fetch('http://localhost:3000/api/produk');
    const response = await res.json();

    return {
        props: {
            products: response.data, // Pass the products data to the page component
        }
    };
}
```

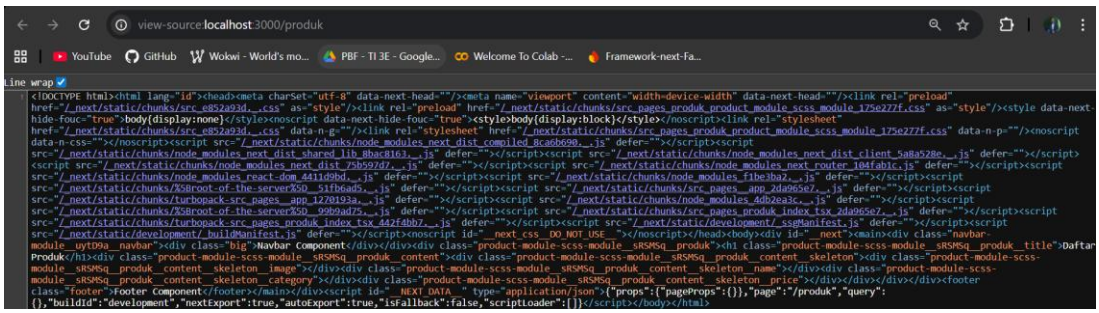
## 2. Dokumentasikan:

- Screenshot CSR
- Screenshot SSR
- Perbedaan Network tab
- Perbedaan View Source

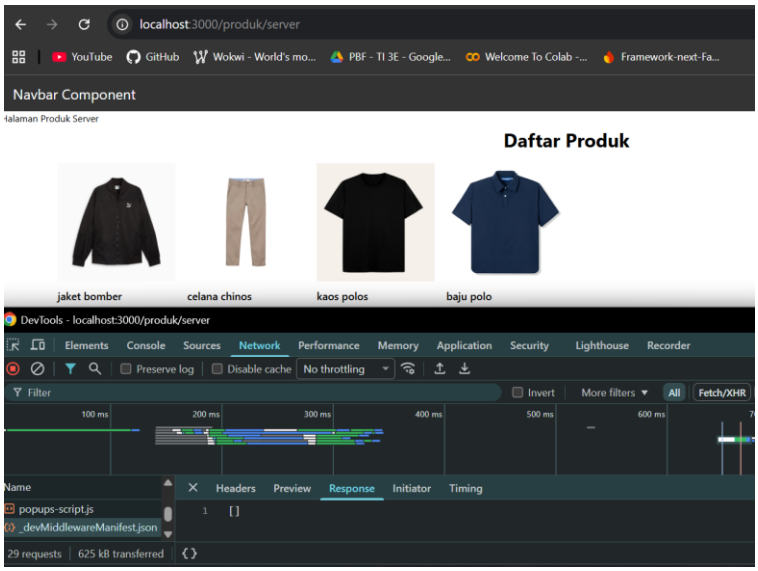
CSR Network:



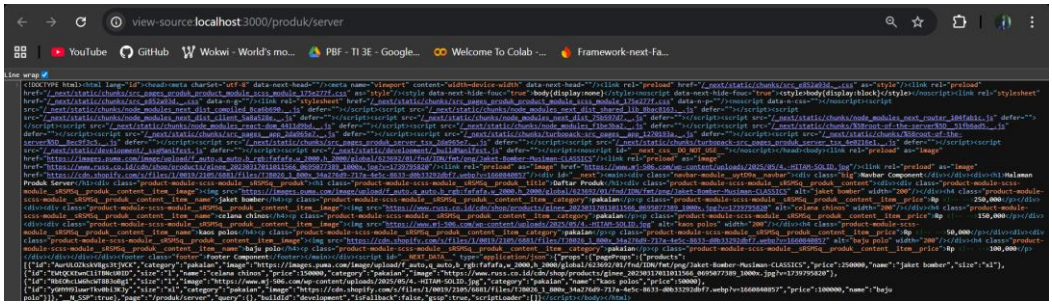

CSR View Source:




SSR Network:




SSR View Source:



## Pertanyaan Refleksi

### 1. Mengapa SSR lebih baik untuk SEO?

Jawab: SSR lebih optimal untuk SEO karena proses *rendering* dilakukan di server. Server menghasilkan dan mengirimkan HTML yang sudah terisi data secara lengkap ke browser, sehingga konten dapat langsung dibaca

### 2. Kapan sebaiknya menggunakan SSR?

Jawab: SSR cocok digunakan ketika aplikasi membutuhkan dukungan SEO dan ketika data harus diambil setiap kali ada permintaan (*request*) baru dari pengguna

### 3. Apa kekurangan SSR dibanding CSR?

Jawab: SSR mengharuskan *rendering* dilakukan di sisi server dan fungsi (seperti getServerSideProps) akan selalu mengambil data dari API sebelum merender halaman setiap kali diakses

### 4. Mengapa skeleton tidak muncul pada SSR?

Jawab: Skeleton tidak muncul dan tidak diperlukan karena HTML sudah dalam keadaan lengkap saat diterima oleh klien. Respons HTML tersebut sudah berisi data produk yang siap ditampilkan